

Unit 1

Lesson 4

Functions in C++

Objective

In this lesson You will get acquainted with an important concept of OOPS i.e.

- Functions
- Call by value
- Call by reference
- Friend function .

In the previous section we have seen how a class is implemented along with the usage of objects and its subsequent components. Now comes the most important building block of the program.... Function. You will understand how important role a Function plays in program through the sessions below.

FUNCTION IN C++

Using functions we can structure our programs in a more modular way, accessing all the potential that structured programming in C++ can offer us.

A function is a block of instructions that is executed when it is called from some other point of the program. The following is its format:

type name (argument1, argument2, ...) statement

where:

- ***type*** is the type of data returned by the function.
- ***name*** is the name by which it will be possible to call the function.
- ***arguments*** (as many as wanted can be specified). Each argument consists of a type of data followed by its identifier, like in a variable declaration (for example, int x) and which acts within the function like any other variable. They allow passing parameters to the function when it is called. The different parameters are separated by commas.
- ***statement*** is the function's body. It can be a single instruction or a block of instructions. In the latter case it must be delimited by curly brackets {}.

function example:

```
//function example  
#include <iostream.h>
```

```
int addition (int a, int b)  
{  
    int r;  
    r=a+b;  
    return (r);  
}
```

```
int main ()  
{  
    int z;  
    z = addition (5,3);  
    cout << "The result is " << z;  
    return 0;  
}
```

output

The result is 8

Function **addition** declares a new variable (**int r;**), and by means of the expression **r=a+b;**, it assigns to **r** the result of **a** plus **b**. Because the passed parameters for **a** and **b** are **5** and **3** respectively, the result is **8**.

The following line of code:

return (r);

finalizes function **addition**, and returns the control back to the function that called it (**main**) following the program from the same point at which it was interrupted by the call to **addition**. But additionally, **return** was called with the content of variable **r** (**return (r);**), which at that moment was 8, so this value is said to be **returned** by the function.

And here is another example about functions:

//function example

```
#include <iostream.h>
```

```
int subtraction (int a, int b)
```

```
{  
    int r;  
    r=a-b;  
    return (r);  
}
```

```
int main ()
```

```
{  
    int x=5, y=3, z;  
    z = subtraction (7,2);  
    cout << "The first result is " << z << "\n";  
    cout << "The second result is " << subtraction (7,2) << "\n";  
    cout << "The third result is " << subtraction (x,y) << "\n";  
    z= 4 + subtraction (x,y);  
    cout << "The fourth result is " << z << "\n";  
    return 0;  
}
```

The first result is 5

The second result is 5

The third result is 2

The fourth result is 6

Arguments passed by *value* and by *reference*..

Until now, in all the functions we have seen, the parameters passed to the functions have been passed *by value*. This means that when calling a function with parameters, what we have passed to

the function were values but never the specified variables themselves. For example, suppose that we called our first function **addition** using the following code :

```
int          x=5,          y=3,          z;  
z = addition ( x , y );
```

What we did in this case was to call function **addition** passing the values of **x** and **y**, that means **5** and **3** respectively, not the variables themselves.

```
int addition (int a, int b)  
           ↑   ↑  
           5   3  
z = addition ( x , y );
```

This way, when function **addition** is being called the value of its variables **a** and **b** become **5** and **3** respectively, but any modification of **a** or **b** within the function **addition** will not affect the values of **x** and **y** outside it, because variables **x** and **y** were not passed themselves to the the function, only their values.

But there might be some cases where you need to manipulate from inside a function the value of an external variable. For that purpose we have to use *arguments passed by reference*, as in the function **duplicate** of the following example:

```
// passing parameters by reference  
#include <iostream.h>  
  
void duplicate (int& a, int& b, int& c)  
{  
    a*=2;  
    b*=2;  
    c*=2;  
}  
  
int main ()  
{  
    int x=1, y=3, z=7;  
    duplicate (x, y, z);  
    cout << "x=" << x << ", y=" << y << ", z=" << z;  
    return 0;  
}
```

```
}
```

output

x=2, y=6, z=14

The first thing that should call your attention is that in the declaration of **duplicate** the type of each argument was followed by an *ampersand* sign (**&**), that serves to specify that the variable has to be passed *by reference* instead of *by value*, as usual.

When passing a variable *by reference* we are passing the variable itself and any modification that we do to that parameter within the function will have effect in the passed variable outside it.

```
void duplicate (int& a,int& b,int& c)
               ↑x   ↑y   ↑z
duplicate (  x  ,  y  ,  z  );
```

To express it another way, we have associated **a**, **b** and **c** with the parameters used when calling the function (**x**, **y** and **z**) and any change that we do on **a** within the function will affect the value of **x** outside. Any change that we do on **b** will affect **y**, and the same with **c** and **z**.

That is why our program's output, that shows the values stored in **x**, **y** and **z** after the call to **duplicate**, shows the values of the three variables of **main** doubled.

If when declaring the following function:

```
void duplicate (int& a, int& b, int& c)
```

we had declared it thus:

```
void duplicate (int a, int b, int c)
```

that is, without the *ampersand* (**&**) signs, we would have not passed the variables *by reference*, but their values, and therefore, the output on screen for our program would have been the values of **x**, **y** and **z** without having been modified.

Passing by reference is an effective way to allow a function to return more than one single value. For example, here is a function that returns the previous and next numbers of the first parameter passed.

FRIENDLY FUNCTIONS

I

We have been emphasizing throughout this chapter that the private members cannot be accessed from outside the class. That is, a non-member function cannot have an access to private data of a class. However, there could be a situation where we would like two classes to share a particular function. For example, consider a case where two classes, manager and scientist, have been defined. We would like to use a function `income_tax( )` to operate the objects of both these classes. In such situations, C++ allows the common function to be made friendly with both the classes, thereby allowing the function to have access to private data of these classes. Such a function need not be a member of any of these classes.

To make an outside function “friendly” to a class, we have to simply declare this function as a friend of the class as shown below

```
Class ABC
{
    .....
    .....
public:
    .....
    .....
    friend void xyz(void);    //declaration
};
```

The function declaration should be preceded by the keyword **friend**. The function is defined elsewhere in the program like a normal C++ function. The function definition does not use either the keyword **friend** or the scope operator `::`. The functions that are declared with the keyword **friend** are known as friend functions. A function can be declared as a **friend** in any number of classes. A friend function, although not a member function, has full access rights to the private members of the class.

Certain special characteristics of friend function :

- It is not in the scope of the class to which it has been declared as friend.
- Since it is not in the scope of the class, it cannot be called using the object of that class.

- It can be invoked like a normal function without the help of any object.
- Unlike member functions, it cannot access the member names directly and has to use object name and dot membership operator with each member name.(e.g. A.x).
- It can be declared either in the public or the private part of a class without affecting its meaning.
- Usually, it has the objects as arguments.

Example : Friend Function

```
#include <iostream>
```

```
using namespace std;
```

```
class sample
```

```
{
```

```
    int a;
```

```
    int b;
```

```
public :
```

```
void setvalue( )
```

```
{
```

```
    a=25; b= 40;
```

```
}
```

```
friend float mean(sample s);
```

```
};
```

```
float mean(sample s)
```

```
{
```

```
    return float(s.a + s.b)/2.0;
```

```
int main( )
```

```
{
```

```
    sample x;    // object X
```

```
    x.setvalue( );
```

```
    cout <<" Mean value = " << mean (x) << "\n";
```

```
    return 0;
```

The output of Program

Mean value = 32.5

The friend function accesses the class variables a and b by using the dot operator and the object passed to it. The function call mean(x) passes the object x by value to the friend function.

Member functions of one class can be friend functions of another class. In such cases, they are defined using the scope resolution operator, which is shown as follows:

```
class x
{
.....
.....
int fun1( );           // member function of X
.....
};

class Y
{
.....
.....

friend int X :: fun1( );      // fun1 () of X
.....                      // is friend of y

};
```

The function **fun1()** is a member of **class X** and a **friend** of **class Y**.

We can also declare all the member functions of one class as the member functions of another class. In such cases, the class is called a friend class. This can be specified as follows:

```
class Z
{
.....
friend class X;      // all member functions of X are
                     // friends to Z
};
```

program demonstrates how member functions work as a bridge between the classes.

```
#include <iostream>
```

```
using namespace std;
```

```
class ABC;           //forward declaration
```

```
class XYZ
{
int x;
public :
    void setvalue(int i)
    {
        x = i;
    }
friend void max(XYZ, ABC);
};
```

```
class ABC
{
```

```

int a;
public :
    void setvalue(int i)
    {
        x = i;
    }
    friend void max(XYZ, ABC);
};

void max(XYZ m, ABC n)           //definition of friend
{
    if(m.x >= n.a)
        cout<<m.x;
    else
        cout<<n.a;
}

int main( )
{
    ABC abc;
    Abc.setvalue(10);
    XYZ xyz;
    xyz.setvalue(20);
    max(xyz, abc);
    return 0;
}

```

output

20

The function max() has arguments from both XYZ and ABC. When the function max() is declared as a friend in XYZ for the first time, the compiler will not acknowledge the presence of ABC unless its name is declared in the beginning as

```
class ABC;
```

This is known as 'forward' declaration.

As pointed out earlier, a friend function can be called by reference. In this case, local copies of the objects are not made. Instead, a pointer to the address of the object is passed and the called function directly works on the actual object used in the call.

This Method can be used to alter the values of the private member of a class. Remember, altering the values of private members is against the basic principles of data hiding. It should be used only when absolutely necessary.

Summary

- A function is a block of instructions that is executed when it is called from some other point of the program.
- Passing by reference is an effective way to allow a function to return more than one single value.
- C++ allows the common function made friendly with both the classes, thereby allowing the function to have access to private data of these classes. Such a function need not be a member of any of these classes.

Questions

1. What is friend function? Explain its characteristic
2. State the merits and demerits of friend function
3. How member function of a class is defined?
4. Write a program for swapping the values of two variables?
5. Write a program to calculate Fibonacci series?
6. What is an advantage of 'call by reference' over 'call by value'?